

Watchdog Timer

Watchdog Timer

1. Watchdog Overview
 - How Important is Watchdog? — Lessons from the Real World
2. Core Concepts
 - 2.1 WDT Types on ESP32-S3
 - 2.2 Task WDT Working Principle
 - 2.3 Timeout Configuration
 - 2.4 Button Falling Edge Detection
3. Project Architecture and Flow
4. Hardware Dependencies
5. Source Code Analysis
 - 5.1 Macro Definitions
 - 5.2 boot_pressed() — Button Read
 - 5.3 Phase 1 — Auto-feed
 - 5.4 Phase 2 — Manual Feed + Countdown (Core)
 - 5.5 Phase 3 — Timeout Reset
6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Output
 - 6.3 Interactive Testing
7. Common Issues

1. Watchdog Overview

Watchdog Timer (WDT) is a hardware safety mechanism. It acts like a "countdown bomb": the system must periodically **feed (reset)** the countdown before timeout, otherwise WDT determines the system has crashed and automatically triggers system reset.

This solves the most common failure mode in embedded systems—program runaway, deadlock, infinite loop. Even if software crashes, hardware WDT can still run independently and bring system back to normal state.

How Important is Watchdog? — Lessons from the Real World

Watchdog is not a "premium feature" but the **bottom line for survival** in embedded products. Let's look at some real scenarios:

- **Industrial Controllers (PLC):** A controller on the production line crashes, the entire assembly line stops, losing potentially tens of thousands of yuan per minute. With WDT, automatic restart and recovery within seconds; without WDT, can only wait for worker to run over and press reset—if no one is around, losses are tens of thousands per hour.
- **Smart meters (electricity/water):** Installed outdoors in pipe wells or on utility poles, once it crashes no one knows, a month of data is lost. WDT is its only way to "self-rescue".
- **Drone flight control:** Program flies away while in the air, without WDT it's free fall; with WDT at least can instantly restart flight control and switch to safe mode landing.

- **Medical equipment:** Monitors, infusion pumps crash, that's not a "just restart" problem—it's a matter of life and death. WDT is a **hard requirement** for medical certification.
- **Automotive ECU:** Engine control unit freezes while driving? Without WDT you wouldn't dare think about it. So automotive MCU watchdog uses independent clock source, completely decoupled from main CPU.
- **Smart home gateway:** Installed in ceiling interstitial space, crashes and no one can reach it. Without WDT it's just a brick, user can only power cycle—terrible experience.

In short: As long as your device isn't connected to a debugger everywhere, you should enable watchdog. Watchdog isn't for preventing bugs, but for preventing the consequences of bugs—bricked devices, production line stoppages, even personal injury. It's "worst-case insurance".

This experiment is divided into three phases:

Phase	Content	Expected Result
Phase 1	Auto-feed 5 times (1s interval)	System runs normally, WDT doesn't trigger
Phase 2	Stop auto-feed, manually feed via BOOT button, screen shows countdown	Each button press resets countdown
Phase 3	Countdown reaches zero, stop feeding	WDT timeout → system auto-reset

2. Core Concepts

2.1 WDT Types on ESP32-S3

ESP-IDF provides two types of watchdogs:

Type	Monitors	Default State	This Experiment
Task WDT	Monitors if registered tasks feed properly	Default ON , auto-monitors idle task	✅ Using
Interrupt WDT	Monitors if ISR occupies CPU too long	Default ON	—

Core idea of Task WDT: If a task doesn't release CPU for a long time (like dead loop, deadlock), WDT times out and resets system. Healthy FreeRTOS tasks periodically call blocking APIs (like `vTaskDelay`), during which idle task automatically feeds watchdog.

2.2 Task WDT Working Principle

```
Task registered to WDT:
    esp_task_wdt_add(NULL) // NULL = current task
    → This task enters WDT monitoring list

Feeding (reset countdown):
    esp_task_wdt_reset() // Reset countdown for all registered tasks
    → Must be called periodically before timeout

Timeout trigger:
    No one feeds within timeout (default 5 seconds)
    → Trigger system reset (equivalent to pressing RESET button)
```

What does registration to WDT mean? System expects this task not to "sleep" longer than timeout. If task executes `vTaskDelay(pdMS_TO_TICKS(6000))` with 5 second timeout, it can't feed during the delay → WDT timeout → reset. Other unregistered tasks are not affected.

2.3 Timeout Configuration

Set in `sdkconfig` (default 5 seconds):

```
CONFIG_ESP_TASK_WDT_TIMEOUT_S=5
```

Or in `menuconfig` → `Component config` → `Common task settings` → `Task watchdog timeout (seconds)`.

2.4 Button Falling Edge Detection

In Phase 2, manual feeding requires detecting BOOT button's **falling edge** (released→pressed moment), not simple low level detection:

```
bool pressed = boot_pressed(); // Current state (true=pressed)
// Falling edge: released→pressed
if (pressed && !last_state) {
    // Feed + reset countdown
}
last_state = pressed;
```

Why use falling edge instead of level detection? If only detecting low level: while holding BOOT button, every loop cycle (50ms) would detect low level and feed, countdown would never reach zero. Falling edge ensures "one press only feeds once", only after release and re-press does it feed again.

3. Project Architecture and Flow

```
app_main()
├─ gpio_config(BOOT_PIN)      Configure BOOT button (GPIO0) input+pull-up
├─ esp_task_wdt_add(NULL)     Register current task to WDT
├─
└─ Phase 1 – Auto-feed (5 times)
```

```

|   for(i=1..5):
|       esp_task_wdt_reset()           Feed
|       vTaskDelay(1000ms)
|
|─ Phase 2 – Manual feed + countdown
|   while(remaining > 0):
|       |─ boot_pressed()             Detect BOOT button
|       |─ Falling edge detected? → feed + reset remaining=5
|       |─ Print countdown every second
|       |─ vTaskDelay(50ms)           50ms polling cycle
|
|─ Phase 3 – Timeout reset
    ESP_LOGE("Watchdog timeout! Resetting system...")
    vTaskDelay(200ms)                 wait for serial flush
    esp_restart()                     Software reset

```

4. Hardware Dependencies

Hardware Connection:

Pin	Peripheral	Configuration	Description
GPIO0	BOOT Button	Input + internal pull-up	Manual feed trigger button

Dependencies: freertos, esp_task_wdt, esp_system, driver/gpio, esp_log

5. Source Code Analysis

5.1 Macro Definitions

This section puts all adjustable numbers at the very top—how many times to feed, interval, countdown seconds, button polling frequency. Later code just sees these uppercase names, changing parameters doesn't require digging to the bottom.

```

#define TAG           "WDT_DEMO"           // Log tag
#define FEED_COUNT     5                   // Auto-feed count
// Feed interval (ms)
#define FEED_INTERVAL_MS 1000
// Countdown after stop (s)
#define COUNTDOWN_SEC  5
#define BOOT_PIN       GPIO_NUM_0         // BOOT button
// Button polling interval (ms)
#define POLL_INTERVAL_MS 50
// Polls per second
#define TICKS_PER_SEC   (1000 / POLL_INTERVAL_MS)

```

`TICKS_PER_SEC = 1000 / 50 = 20`—every 20 polls (50ms × 20 = 1000ms) subtract one second from countdown.

5.2 boot_pressed() — Button Read

One line: read BOOT pin with `gpio_get_level`, low level (0) means button pressed, return `true`. Because GPIO0 is configured with internal pull-up, released is high level.

```
static bool boot_pressed(void)
{
    // Pull-up, LOW=pressed
    return gpio_get_level(BOOT_PIN) == 0;
}
```

5.3 Phase 1 — Auto-feed

Program first proves its innocence—loop 5 times, each time feed once then sleep 1 second. If all 5 runs complete normally, WDT functions properly and won't accidentally trigger reset.

```
for (int i = 1; i <= FEED_COUNT; i++) {
    esp_task_wdt_reset();                // Feed WDT
    ESP_LOGI(TAG, "Feed #%d", i);
    vTaskDelay(pdMS_TO_TICKS(FEED_INTERVAL_MS)); // wait 1s
}
```

This phase verifies **WDT basic function is normal**: periodic feeding → system runs stably.

5.4 Phase 2 — Manual Feed + Countdown (Core)

This is the main event. No auto-feed, instead scan BOOT button every 50 milliseconds—press once, feed once, countdown resets to 5; don't press, countdown decrements every second, reaches 0 then enters Phase 3. Key is "falling edge detection" (previous released, current pressed counts as one), holding doesn't count as cheating.

```
int remaining = COUNTDOWN_SEC;
int tick      = 0;
bool last_state = false;

while (remaining > 0) {
    bool pressed = boot_pressed();

    // Falling edge detect – feed once per press, long press doesn't repeat
    if (pressed && !last_state) {
        esp_task_wdt_reset();                // Feed WDT
        ESP_LOGI(TAG, ">>> Boot feed! WDT reset <<<");
        // Reset countdown
        remaining = COUNTDOWN_SEC;
        tick      = 0;
    }
    last_state = pressed;

    // Print every second
    if (tick % TICKS_PER_SEC == 0)
        ESP_LOGW(TAG, "Countdown %d ...", remaining);
}
```

```

vTaskDelay(pdMS_TO_TICKS(POLL_INTERVAL_MS));    // 50ms polling

// Every 20 polls = 1 second
if (++tick >= TICKS_PER_SEC) {
    tick = 0;
    remaining--;
}
}

```

Time Precision Note:

- 50ms polling cycle × 20 = 1000ms/second, countdown precision ±50ms
- `remaining` only decrements when `tick` accumulates to 20, ensuring each countdown second truly passes ~1 second

5.5 Phase 3 — Timeout Reset

Countdown reached 0, meaning no one fed. First print an error log, wait 200ms for serial to finish outputting, then actively call `esp_restart()` to reset system—effect equivalent to pressing RESET button, program starts over from beginning. Why not wait for WDT natural timeout? Because that has uncertain delay, actively restarting is cleaner.

```

ESP_LOGE(TAG, "Watchdog timeout! Resetting system...");
// wait 200ms for log flush
vTaskDelay(pdMS_TO_TICKS(200));
esp_restart();                // Trigger system reset

```

Why `vTaskDelay(200ms)` before calling `esp_restart()` instead of waiting for WDT natural timeout? There's uncertain delay between WDT timeout and reset (depends on interrupt handling). Actively calling `esp_restart()` ensures log is completely flushed to serial before reset, observers can see "Resetting system..." message.

6. Operation Flow

6.1 Build and Flash

For detailed flashing process, see: 1. Before Development → 3. Build and Flash

6.2 Normal Output

Phase 1 — Auto-feed (5 seconds):

```

I (xxx) WDT_DEMO: =====
I (xxx) WDT_DEMO: ESP32-S3 Watchdog Test (IDF V6.0.1)
I (xxx) WDT_DEMO: =====
I (xxx) WDT_DEMO: BOOT button (GPIO0) ready – press to feed WDT
I (xxx) WDT_DEMO: Task added to WDT, timeout: 5s
I (xxx) WDT_DEMO: Feed #1
I (xxx) WDT_DEMO: Feed #2
I (xxx) WDT_DEMO: Feed #3
I (xxx) WDT_DEMO: Feed #4
I (xxx) WDT_DEMO: Feed #5

```

```

I (242) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (248) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (255) main_task: Started on CPU0
I (275) main_task: Calling app_main()
I (275) WDT_DEMO: =====
I (275) WDT_DEMO: ESP32-S3 Watchdog Test (IDF V6.0.1)
I (275) WDT_DEMO: =====
I (275) WDT_DEMO: BOOT button (GPIO0) ready – press to feed WDT
I (285) WDT_DEMO: Task added to WDT, timeout: 5s
I (285) WDT_DEMO: Feed #1
I (1295) WDT_DEMO: Feed #2
I (2295) WDT_DEMO: Feed #3
I (3295) WDT_DEMO: Feed #4
I (4295) WDT_DEMO: Feed #5
W (5295) WDT_DEMO: Auto-feeding stopped! Press BOOT to feed...
W (5295) WDT_DEMO: Countdown 5 ...
W (6295) WDT_DEMO: Countdown 4 ...
W (7295) WDT_DEMO: Countdown 3 ...
W (8295) WDT_DEMO: Countdown 2 ...
E (9285) task_wdt: Task watchdog got triggered. The following tasks/users did not reset the watchdog in time:
E (9285) task_wdt: - main (CPU 0)
E (9285) task_wdt: Tasks currently running:
E (9285) task_wdt: CPU 0: IDLE0
E (9285) task_wdt: CPU 1: IDLE1
E (9285) task_wdt: Print CPU 0 (current core) backtrace

```

Phase 2 — Manual Feed + Countdown:

```

W (xxx) WDT_DEMO: Auto-feeding stopped! Press BOOT to feed...
W (xxx) WDT_DEMO: Countdown 5 ...
W (xxx) WDT_DEMO: Countdown 4 ...
I (xxx) WDT_DEMO: >>> Boot feed! WDT reset <<<      ← Pressed BOOT
W (xxx) WDT_DEMO: Countdown 5 ...                    ← Countdown reset to 5
W (xxx) WDT_DEMO: Countdown 4 ...
...

```

```
I (255) main_task: Started on CPU0
I (275) main_task: Calling app_main()
I (275) WDT_DEMO: =====
I (275) WDT_DEMO: ESP32-S3 Watchdog Test (IDF V6.0.1)
I (275) WDT_DEMO: =====
I (275) WDT_DEMO: BOOT button (GPIO0) ready – press to feed WDT
I (285) WDT_DEMO: Task added to WDT, timeout: 5s
I (285) WDT_DEMO: Feed #1
I (1295) WDT_DEMO: Feed #2
I (2295) WDT_DEMO: Feed #3
I (3295) WDT_DEMO: Feed #4
I (4295) WDT_DEMO: Feed #5
W (5295) WDT_DEMO: Auto-feeding stopped! Press BOOT to feed...
W (5295) WDT_DEMO: Countdown 5 ...
I (5645) WDT_DEMO: >>> Boot feed! WDT reset <<<
W (5645) WDT_DEMO: Countdown 5 ...
W (6645) WDT_DEMO: Countdown 4 ...
I (7195) WDT_DEMO: >>> Boot feed! WDT reset <<<
W (7195) WDT_DEMO: Countdown 5 ...
W (8195) WDT_DEMO: Countdown 4 ...
I (9195) WDT_DEMO: >>> Boot feed! WDT reset <<<
W (9195) WDT_DEMO: Countdown 5 ...
W (10195) WDT_DEMO: Countdown 4 ...
W (11195) WDT_DEMO: Countdown 3 ...
I (11245) WDT_DEMO: >>> Boot feed! WDT reset <<<
W (11245) WDT_DEMO: Countdown 5 ...
W (12245) WDT_DEMO: Countdown 4 ...
W (13245) WDT_DEMO: Countdown 3 ...
W (14245) WDT_DEMO: Countdown 2 ...
```

Phase 3 — Timeout Reset:

```
W (xxx) WDT_DEMO: Countdown 1 ...
E (xxx) WDT_DEMO: Watchdog timeout! Resetting system...
--- System reset, restarting ---
```

6.3 Interactive Testing

Operation	Expected
Don't press any key	Countdown 5→4→3→2→1→0 → system reset
Press BOOT during countdown	Immediately feed + countdown reset to 5
Hold BOOT	Only feeds once (falling edge detection), countdown restarts from 5
Rapidly press BOOT	Each falling edge resets countdown

7. Common Issues

Phenomenon	Cause	Solution
Resets during Phase 1	WDT timeout < feed interval / task blocked for long time	Increase <code>CONFIG_ESP_TASK_WDT_TIMEOUT_S</code> (default 5s)
Press BOOT no response	GPIO0 is strapping pin, forced high by pull-up	Normal: button pressed is still low, falling edge detection unaffected

Phenomenon	Cause	Solution
Countdown decreases while holding BOOT	Level detection instead of falling edge detection	Check <code>pressed && !last_state</code> logic
Auto-reset too fast	<code>POLL_INTERVAL_MS</code> or <code>COUNTDOWN_SEC</code> values too small	Confirm <code>COUNTDOWN_SEC=5</code>
<code>esp_task_wdt_add</code> fails	Task already registered / WDT not initialized	Ensure called only once in <code>app_main</code>
Watchdog doesn't trigger reset	<code>esp_restart()</code> proactively resets before WDT timeout	This is the experiment's design! Using <code>esp_restart</code> instead of natural timeout